

TESIS DOCTORALES UPC DL

1. “Analyzing and Leveraging the Structure of Pre-trained Embeddings” (UPC, 2025) <https://upcommons.upc.edu/entities/publication/f7b39214-6b51-4606-8c99-1e084f56107b>

- Analiza **por qué los embeddings preentrenados funcionan** (tipo BERT, encoder-only).
- Demuestra que el rendimiento depende del **alineamiento entre etiquetas/task y estructura del espacio vectorial**.
- Propone métodos para mejorar rendimiento en:
 - few-shot learning
 - baja anotación
 - clasificación semántica

Aplicación directa al proyecto DL:

La pipeline del proyecto incluye:

- BERT-like LLM (encoder-only)
- vec DB
- Retrieval

Esta tesis te ayuda a:

- mejorar cómo generas embeddings antes de indexar.
- optimizar retrieval semántico.
- entender cuándo los embeddings fallan (muy importante en RAG).

2. “Enhancing Word and Document Embeddings for NLP Tasks” (UPC) 2025

<https://upcommons.upc.edu/entities/publication/de5d5c44-712c-46ad-b66b-5eca1b704b63>

- Mejora embeddings a nivel:
 - palabra
 - documento (clave para tus .docx)
- Investiga **representaciones semánticas profundas** para tareas NLP.

Aplicación al proyecto DL

Perfecto para:

- pipeline de ingestión (.docx - tokenizer - embeddings)
- chunking + representación vectorial
- ranking en retrieval

base teórica sólida para la parte **indexing + encoding**

3. “Generación de modelos a partir de documentos de texto” (UPC, 2025)

<https://upcommons.upc.edu/entities/publication/efdce4ac-5007-4447-9951-bd4391112d5b>

- Trabaja con:
 - documentos no estructurados
 - extracción de información

- uso de LLMs (incluidos prompts)
- Usa dos enfoques:
 - NLP clásico
 - LLMs modernos

Aplicación directa al proyecto DL

- ingestion
- chunking semántico
- extracción contextual

Además introduce:

- **prompt design** útil para tu bloque GPT-like
- combinación NLP clásico + LLM : igual que vuestro esquema híbrido

Insights clave aplicados a la arquitectura del proyecto

hay 3 ideas importantes que las tesis confirman:

3.1 Separación encoder (retrieval) vs decoder (generación)

Tu modelo:

- BERT-like - embeddings
- GPT-like - respuesta

Esto es el estándar actual (RAG moderno)

Las tesis: confirman que los embeddings preentrenados son el núcleo del rendimiento.

3.2 El punto crítico NO es el LLM sino que es el embedding + indexing

Problema típico:

“el RAG no responde bien”

En realidad falla:

- chunking
- embedding quality
- retrieval ranking

Las tesis de UPC se centran justo en eso:

- estructura del embedding space
- representación semántica

3.3 Mezclar approaches (NLP + LLM) es clave

El pipeline de proyecto:

- tokenizer + reglas
- embeddings
- LLM

Esto coincide con investigación actual:

- NLP clásico : precisión estructural
- LLM : flexibilidad

Recomendaciones para mejorar la arquitectura del proyecto DL (basado en las tesis):

1. Mejora el Retrieval

- usar:
 1. re-ranking (cross-encoder)
 2. no solo cosine similarity
- explorar:
 1. cluster-aware retrieval (según estructura del espacio)

2. Ajusta embeddings

- domain fine-tuning
- sentence-transformers específicos

3. Mejora chunking

no solo por tamaño sino también por:

- semántica
- estructura documental (.docx)

4. Usa “query decomposition”

Si Q se descompone - subqueries - agregación

Esto es muy avanzado (tipo:

- multi-hop reasoning
- agentic retrieval)

Generación de modelos de procesos y decisiones a partir de documentos de texto

Preguntas:

1. Retrieval + embeddings

a) ¿Cómo representarías entidades como actividades, condiciones o decisiones en el espacio de embeddings para mejorar retrieval?

En la tesis doctoral “Generación de Modelos de Procesos y Decisiones a partir de Documentos de Texto” (UPC, 2025), Autor: Luis Hernan Quishpi Betún [TLHQB1de1.pdf | PDF]

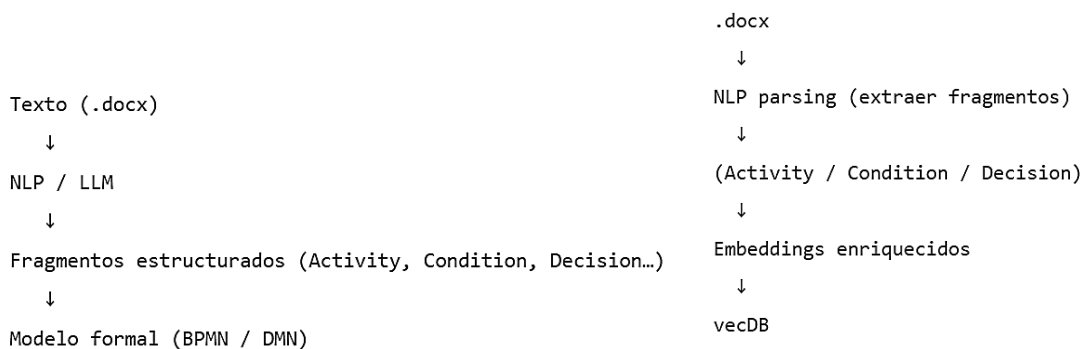
1. La tesis parte de texto no estructurado: Documentos en lenguaje natural (.docx, etc.) Problema: ambigüedad, difícil de procesar.

2. Luego hace extracción de fragmentos semánticos, Ejemplos: actividades, condiciones, decisiones, input data...esto se hace con: NLP (árboles de dependencia + patrones) o LLM + prompting.

3. Luego construye una representación estructurada intermedia:

- ATDP (Annotated Textual Descriptions of Processes).
- fragmentos + relaciones (agent, patient, sequence, conflict...).

4.Finalmente genera modelos formales: BPMN (procesos), DMN (decisiones).



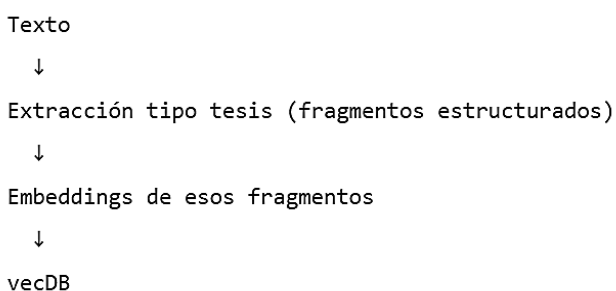
El proyecto:

Texto → embeddings → retrieval → LLM

La tesis propone:

Texto → estructura semántica (antes)

Es decir, en lugar de hacer embeddings de texto plano, hacer:



La tesis demuestra que el valor está en la estructura semántica, no en el texto en bruto. Antes de convertir el texto en vectores, se convierte en estructura semántica, fragmentos tipados.

b) ¿Cómo hacer el retrieval correctamente con embeddings estructurados?

RAG básico hace: Responde parecido, no entiende relaciones, puede mezclar ideas diferentes.

Las queries NO son simples búsquedas (“busca texto relevante”). Son:

- multi-documento
- analíticas (agregación, patrones, ranking)
- orientadas a ejecución de obra

query → extracción dirigida → conjunto estructurado → análisis

Paso 1 — Interpretación de la query

Query 1: “¿Cuáles son las incidencias más frecuentes?”

tipo = action/issue
operación = agregación (count + clustering)
scope = multi-documento

Query 14: “¿Cuáles son las acciones pendientes más frecuentes?”

tipo = action
filtro = estado: pendiente
operación = agregación

Query 6: “¿Qué obras presentan mayor riesgo?”

tipo = issue + condition
operación = scoring
nivel = obra

Se hace con LLM classifier (ideal) o reglas + vocabulario técnico.

Paso 2 — Retrieval selectivo por tipo. No se busca todo. Según query:

Query	Retrieval
Incidencias	type = action/issue
Pendientes	type = action + estado pendiente
Riesgos	type = condition + hazard
Instalaciones	type = entity + instalación
Normativa	type = condition + no cumple

Ejemplo real

filter:

type = action

categoria = incidencia

Paso 3 — Retrieval masivo (NO top-k)

Las queries necesitan top 100 o filtered set completo

Paso 4 — Ranking inteligente

Paso 5 — Post-retrieval

A. Clustering semántico

Query:

“incidencias más frecuentes”

pipeline:

1. recuperar 50 incidencias
2. clustering embeddings
3. agrupar

Resultado:

- filtraciones (34)
- retrasos eléctricos (21)
- defectos estructurales (18)

B. Aggregation / ranking

Query:

“ranking de elementos con más problemas”

pipeline:

1. recuperar incidencias
2. mapear → entity
3. contar

Resultado:

1. andenes (45)
2. luminarias (32)
3. accesos (28)

C. Timeline reasoning

Query:

“cómo evolucionan incidencias”

pipeline:

1. recuperar incidencias
2. ordenar por fecha
3. detectar tendencia

D. Risk scoring

Query:

“qué obras tienen mayor riesgo”

pipeline:

1. recuperar incidencias por obra
2. scoring:

risk_score =

nº incidencias

+ gravedad

+ seguridad

8. Multi-hop retrieval (clave en tu caso)

Muchas queries requieren varios pasos:

Query 8:

“problemas que afectan a seguridad ferroviaria”

flujo:

1. buscar condiciones: → "riesgo", "afección", "seguridad"
2. recuperar incidencias asociadas
3. agrupar

esto NO es un único retrieval

9. Hybrid retrieval (OBLIGATORIO aquí)

Porque los datos tienen lenguaje técnico

vector search

para:

- “filtración” ≈ “humedad”

keyword

para:

- “andenes”
- “luminarias”
- “IP65”

combinado:

score = 0.7 vector + 0.3 keyword

10. Arquitectura FINAL

User Query



Query interpreter (tipo + operación)



Retrieval plan:

- qué tipo buscar
- filtros metadata



Vector DB:

- retrieval masivo (no top-k)
- filtrado previo



Post-processing:

- clustering
- agregación
- ranking
- timeline



LLM (solo para síntesis)



Respuesta final

c) ¿Cómo evitar pérdida de información estructural cuando usamos embeddings (ya que aplanan la jerarquía del texto)?

Para evitar pérdida de estructura:

1. separar texto en fragmentos
2. tiparlos (incidencia, acción...)
3. añadir metadata rica
4. usar embeddings SOLO para semántica
5. modelar relaciones fuera del embedding
6. hacer retrieval híbrido (estructura + vector)

Chunking inteligente (problema crítico en RAG)

- d) ¿Cómo definirías chunking semántico acorde a los límites de actividades/procesos en lugar de longitud fija?
- ¿Usarías dependency parsing para fragmentar el texto antes de indexar?
 - ¿Cómo tratar casos inter-oración donde una actividad depende de otra (context windows vs retrieval)?

Problema detectado en tesis:

relaciones **inter-oración son complejas y críticas**

chunk = 500 tokens

Puede romper completamente: incidencias a medias, acciones separadas de su contexto y pérdida de estructura técnica.

Un chunk = unidad semántica operativa completa

Tipos de chunk en actas

Cada acta se descompone en:

1. Incidencia / problema

"Se detecta filtración en cubierta"

2. Acción / tarea

"El contratista debe corregir la filtración"

3. Estado / condición

"Instalación pendiente de revisión"

4. Decisión técnica

"Se decide sustituir material"

5. Evento temporal / seguimiento

"En la visita anterior se corrigió..."

¿Usar dependency parsing para fragmentar?

Dependency parsing te permite:

- identificar sujetos : quién actúa
- verbos : acción/incidencia
- objetos : elemento afectado

Dónde usarlo

A. Detectar unidades semánticas

Ejemplo:

"El contratista revisa la cubierta y corrige filtraciones"

Parsing detecta:

- acción 1 : revisar cubierta
- acción 2 : corregir filtraciones

genera 2 chunks (clave)

B. Extraer metadata automáticamente

De:

"Se detecta filtración en la cubierta"

extraes:

type = incidencia

zona = cubierta

acción = detectar

Dependency parsing SOLO no:

- falla con lenguaje técnico imperfecto
- no detecta bien:
 - estados ("pendiente")
 - categorías (riesgo, normativa...)

SOLUCIÓN REAL

Usar:

Parsing + reglas + LLM (ligero)

pipeline óptimo:

Texto

↓

Dependency parsing

↓

Reglas dominio (obra)

↓

Enriquecimiento con LLM

↓

Chunk estructurado

¿Cómo tratar dependencias inter-oración?

Problema

Ejemplo real:

Se detecta filtración en cubierta.

Se solicita al contratista su reparación.

relación:

incidencia → acción correctiva

Si haces chunking puro:

- pierdes esa relación
- retrieval devuelve cosas sueltas

Soluciones

Opción 1 — Metadata relacional

Cada chunk guarda referencias:

JSON

```
{  
  "type": "action",  
  "text": "reparar filtración",  
  "relaciona_con": "filtración en cubierta"  
}
```

4. Extracción y estructuración (capa previa al LLM)

- ¿Qué ventajas aporta usar parsing sintáctico y roles semánticos antes del embedding?
- ¿Podría el uso de SRL (agent/patient) mejorar el retrieval al enriquecer contexto?
- ¿Cómo decidir qué información estructural preservar antes de pasar al LLM?
- ¿Crees que generar primero un modelo intermedio (ATDP) mejora la robustez del sistema?

Base tesis:

- extracción de roles, actividades, condiciones, relaciones

5. Uso de LLM (vuestro bloque GPT-like)

- ¿Qué estrategia recomiendas: prompting puro vs fine-tuning vs hybrid (RAG + prompts + reglas)?

- ¿Cómo estructurar prompts para extraer decisiones o procesos sin que el modelo “alucine”?
- ¿Recomiendas chain-of-thought o structured outputs para este tipo de tareas?
- ¿Cómo validar automáticamente que la salida del LLM es consistente (tipo BPMN/DMN)?

Tesis:

- LLM = flexible pero menos explicable y controlable

6. NLP vs LLM (pregunta estratégica)

Esta es importante para orientar vuestro proyecto:

- ¿Qué parte del pipeline debería quedarse en NLP clásico y cuál migrar a LLM?
- ¿Hay un “sweet spot” híbrido óptimo (ej: NLP para parsing + LLM para inferencia)?
- ¿Cómo medir objetivamente cuándo usar cada enfoque? (precisión, F1, robustez)

Tesis:

- compara ambos enfoques y sus trade-offs (precisión vs flexibilidad)

7. Evaluación y métricas (muy importante para proyecto)

- ¿Qué métricas usarías en un sistema RAG aplicado a procesos? (¿más allá de precisión textual?)
- ¿Cómo evaluar calidad de retrieval vs calidad de generación?
- ¿Tiene sentido evaluar a nivel de modelo BPMN/DMN en lugar de texto?
- ¿Cómo diseñar un benchmark realista con documentos empresariales?

Tesis usa:

- precision / recall / F1 para extracción

8. Problemas difíciles (para discusión avanzada)

- ¿Cómo resolver ambigüedad semántica en lenguaje natural antes del modelado?
- ¿Qué técnicas recomiendas para coreference resolution a escala empresarial?
- ¿Cómo tratar inconsistencias entre documentos (knowledge conflict)?
- ¿Cómo detectar que un modelo generado es incompleto o incorrecto?

Problema central tesis:

- ambigüedad del lenguaje natural

9. Arquitectura

- ¿Cómo diseñarías una arquitectura completa combinando: parsing - embeddings - retrieval - LLM?
- ¿Dónde introducirías caching, re-ranking o validación?

- ¿Se puede optimizar el pipeline con feedback loops (LLM → mejora retrieval)?
- ¿Cómo escalar esto a grandes volúmenes documentales?

10. Futuro / mejoras (preguntas de investigación)

- ¿Qué líneas de investigación ves más prometedoras en este campo?
- ¿Tiene sentido incorporar agentes autónomos en el pipeline?

Para preguntas Queries complejas.

Query:

“¿Cuáles son las incidencias más frecuentes en todas las actas?”

```
[User Query]
  ↓
[Agent 1: Query Planner]
  ↓
[Engine determinista (retrieval + DB)]
  ↓
[Agent 2: Analysis Agent]
  ↓
[Agent 3: Report Agent]
  ↓
[Respuesta]
```

1. Agent 1 — Query Planner (EL MÁS IMPORTANTE)

Este agente interpreta y descompone la query

INPUT

“¿Cuáles son las incidencias más frecuentes?”

OUTPUT del agente

```
{
  "tipo": "incidencias",
  "operaciones": [
    "retrieve",
    "cluster",
    "count",
    "rank"
  ],
  "filtros": {
    "scope": "todas las obras"
  }
}
```

convierte lenguaje natural en plan ejecutable

Agente	Función
Query Planner	entiende y descompone
Analysis Agent	interpreta datos
Report Agent	redacta respuesta

- ¿Qué ventajas hay en la utilización de GRAPH?

Porque tus queries son de tipo:

- patrones
- relaciones
- agregaciones
- dependencias

Ejemplos:

- “qué elementos tienen más incidencias”
- “qué obras presentan mayor riesgo”
- “qué acciones están pendientes”
- “patrones recurrentes entre obras” [

Esto NO es texto, es estructura

Insight clave

Un grafo representa relaciones explícitas que los embeddings SOLO capturan implícitamente

Cómo modelar un grafo para actas de obra

NODOS = entidades reales del sistema

ARISTAS = relaciones entre ellas

Nodos

- Obra
- Acta
- Incidencia
- Acción
- Elemento constructivo
- Responsable
- Fecha

Relaciones

(Obra) — tiene —> (Acta)
(Acta) — contiene —> (Incidencia)
(Incidencia) — afecta —> (Elemento)
(Incidencia) — genera —> (Acción)
(Acción) — asignado_a —> (Responsable)
(Acción) — estado —> pendiente/resuelto

Ejemplo real

Acta:

“Se detecta filtración en cubierta.

El contratista debe repararla.”

Grafo:

[Incidencia: filtración]

↓ afecta

[Elemento: cubierta]

[Incidencia] → genera → [Acción: reparación]

↓

[Responsable: contratista]

Cómo responde el grafo a las queries

Query:

“¿Cuáles son las incidencias más frecuentes?”

Graph query (tipo Neo4j / Cypher):

MATCH (i:Incidencia)

RETURN i.tipo, COUNT(*)

ORDER BY COUNT DESC

resultado inmediato (sin LLM)

Query:

“¿Qué elementos tienen más problemas?”

MATCH (i:Incidencia)-[:AFECTA]->(e:Elemento)

RETURN e.nombre, COUNT(*)

ORDER BY COUNT DESC

ranking directo

Query:

“¿Qué acciones pendientes hay?”

MATCH (a:Accion {estado:"pendiente"})

RETURN a

trivial para grafo

Query:

“¿Qué obras presentan mayor riesgo?”

MATCH (o:Obra)-[:TIENE]->(a:Acta)-[:CONTIENE]->(i:Incidencia)

RETURN o, COUNT(i) as riesgo

ORDER BY riesgo DESC

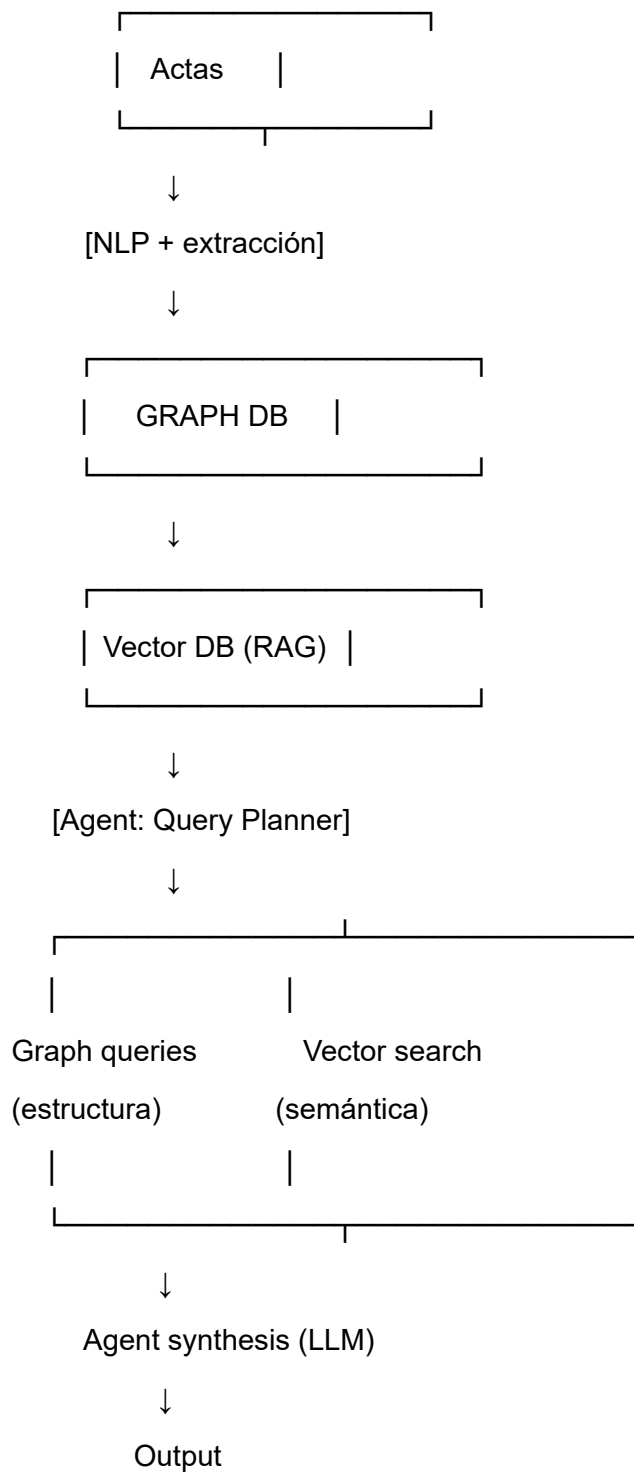
OK

Comparativa directa

Enfoque	Pros	Contras
RAG	flexible	débil en estructura
Agentes	buenos para reasoning	no escalan en datos
Graph	perfecto para relaciones	menos semántico

Arquitectura

Graph + RAG + Agents



Graph, SIEMPRE para:

- contar incidencias
- agrupar
- tendencias

- ranking
- dependencias

Vector DB ,SOLO para:

- lenguaje ambiguo
- detección semántica
- “filtración $\approx$ humedad”

Agent , SOLO para:

- interpretar query
- planificar
- generar informe final

Ejemplo completo (pipeline híbrido)

Query:

“Detecta patrones repetidos de incidencias”

Pipeline:

1. Agent Planner

tipo = incidencia

operación = pattern detection

2. Retrieval

Graph:

incidencias + elementos + obras

3. Analysis

clustering embeddings

4. Output

Agent genera:

"Se repite filtración en cubiertas en varias obras"

Conclusión

Graph es la base estructural (core del sistema)

Vector embeddings complementan semántica

Agentes orquestan y explican

“La solución óptima combina un grafo como fuente estructural principal, embeddings para similitud semántica, y agentes para orquestación y síntesis analítica.”

- **¿Qué falta actualmente en el estado del arte para sistemas como el nuestro?**